KUBERNETES PREPARATION · MY LEARNING NOTES · SESSION 10

StatefulSets — Concepts, ACID, and Headless Service DNS

Your notes, cleaned up from dictation + screenshots — Lecture 1 of the StatefulSets series

Session agenda

1. What are Stateful & Stateless apps
2. Why & What of StatefulSets
3. Demo 1: Implementing StatefulSets on Amazon EKS *(upcoming — not yet covered this session)*
4. Demo 2: Chaos Engineering — Volume Reattachment *(upcoming — not yet covered this session)*

> This session covered **agenda items 1 and 2** (the conceptual foundation) plus a first look at the EKS control-plane/data-plane split that Demo 1 will build on. The two hands-on demos will get their own session file(s) once recorded.

Quick recall

- **Stateful app** = code **+ data**, tightly coupled — behavior depends on internal memory or persistent data it holds/stores.
- **Stateless app** = just code — can be stopped, restarted, or rescheduled with **zero** user disruption.
- A single application is rarely *all* one or the other — different **tiers** of the same app can be stateful or stateless independently.
- **StatefulSet (STS)** = the Kubernetes workload controller purpose-built for the stateful case: stable pod identity, stable network identity, ordered start/stop, and persistent volume re-attachment — none of which a Deployment gives you.

PART 1 · STATEFUL VS. STATELESS, WORKED THROUGH A REAL USER FLOW

The 3-tier breakdown

```
User → Web Tier (frontend) → Application Tier (backend/business logic) → Data Tier (database)
        [stateful -- see below]      [stateless]                          [stateful]
```

- **Web Tier** — the user interface. *(Why this is called out as stateful below — it's the part that surprises people.)*
- **Application Tier** — handles business logic, processing, communication with the database. Just runs code; doesn't hold onto data itself → **stateless**.
- **Data Tier** — stores, manages, and retrieves data (a StatefulSet-backed database, or something like Amazon RDS) → **stateful**, always.

Why the Web Tier can be stateful too — the session-data problem

Worked example: user **XYZ** logs into an Amazon.com-style site, sets country = India, currency = INR, switches on dark mode, and starts browsing a "trimmers" category.

Under the hood, a **session** gets created holding all of that: preferences, currency, dark mode, what he's currently browsing. If that session is stored on the **local filesystem/cache of the one pod** currently serving him:

- User is being served by **Pod 1**, session lives only on Pod 1.
- **Pod 1 dies.** The session data dies with it.
- **Pod 2** picks up the next request — but knows *nothing* about India/INR/dark-mode/trimmers. The user has to re-enter everything. Bad experience, and this is exactly why that Web Tier pod is "stateful" in this architecture — its behavior depends on data only it is holding.

> ⚠ **Conclusion:** a stateless-looking pod becomes *effectively* stateful the moment it holds request-critical data nowhere else — the fix is never "make the pod stateful," it's "move the state somewhere both pods can reach."

The fix — externalize session state to a shared cache

Second architecture: introduce **Amazon ElastiCache** (or equivalent) as a shared session store. Both the Web Tier and Application Tier pods become genuinely **stateless** (plain `Deployment` pods, not `StatefulSet`) — they read/write session data to the shared cache instead of local storage. The Data Tier stays the one truly stateful piece.

```
User → Web Tier (stateless, Deployment) ----+
                                            +---> Amazon ElastiCache (stores session info)
       App Tier (stateless, Deployment) ----+
                     |
                     v
       Data Tier (stateful, StatefulSet or Amazon RDS)
```

Why this is a genuinely better design, not just a workaround:

- **Seamless failover:** Pod 1 dies mid-session → Pod 2 (or any pod) picks up the exact same request — the session data was never tied to a specific pod in the first place.

- **Less load on the real database:** repeated/frequently-read information can be served straight from cache instead of round-tripping to the database every time.
- **Speed:** cache mechanisms store data in **memory**, not on disk — dramatically faster than a database round-trip.

The general rule this teaches: in Kubernetes, **stateless apps scale horizontally with no friction** (no data to reconcile between replicas). **Stateful apps are the tricky part** — and that's specifically what StatefulSets exist to make tractable.

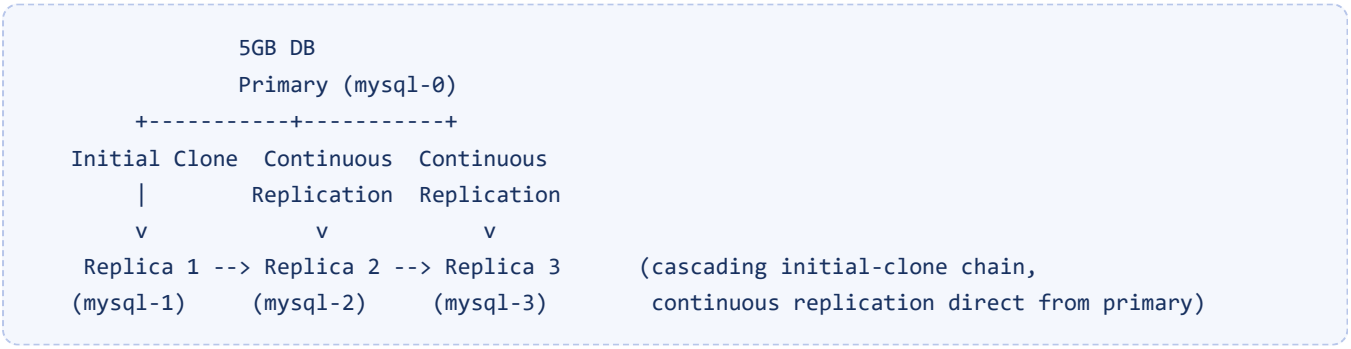
PART 2 · DATABASES ARE ALWAYS STATEFUL: REPLICATION, AND ACID

Primary + replicas

- **One primary node** (formerly "master"): handles **all writes**.
- **Multiple replicas** (read-only): provide redundancy and help with **read scaling**.

Setting up a replica is 2 stages:

1. **Initial Cloning** — copy the existing data (e.g. a 5GB DB) to the new replica. Can be **cascading** (replica clones from another replica, not always straight from primary).
2. **Continuous Replication** — keep syncing every future write/modification from the primary going forward. Can be **direct** (each replica syncs straight from the primary).



Why this architecture exists (2 separate reasons, both matter):

- **Performance:** write-heavy operations go to the primary only; read-heavy operations get offloaded to replicas — this read/write separation scales the system under high concurrency far better than one node doing everything.
- **High availability:** if the primary node fails, **a replica gets promoted to primary**, keeping downtime minimal and service continuous. This is not just a performance optimization — it's a resilience mechanism.

ACID — the relational-database contract this all exists to protect

Letter	Term	Meaning
A	Atomicity	A transaction is all-or-nothing — it succeeds completely or fails completely, never left in an in-between state.
C	Consistency	A transaction moves the database from one **valid** state to another valid state.
I	Isolation	Concurrent transactions don't interfere with each other — each preserves data correctness as if run alone.
D	Durability	Once a transaction is committed, the change is **permanent** — survives even a crash right after.

Why this matters for the infra layer: a plain `Deployment` cannot guarantee any of this for a database, because it doesn't guarantee *which* pod comes back after a failure, *which* disk it reattaches to, or *what order* replicas start in — and all three are load-bearing for keeping replication (and therefore ACID guarantees across the whole cluster) intact.

PART 3 · THE REQUIREMENTS THAT DEFINE A STATEFULSET

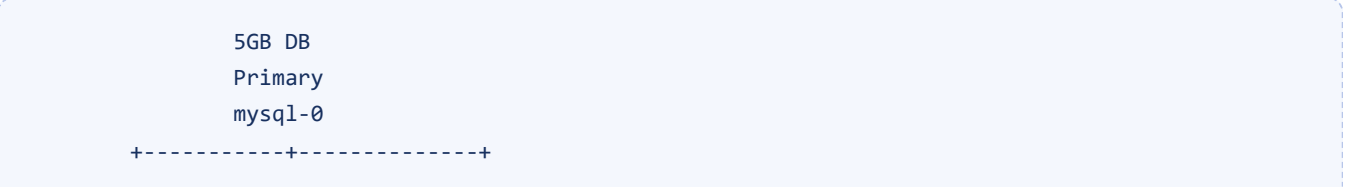
Databases like MySQL/PostgreSQL specifically require:

- **High IOPS and low latency**
- **Stable volumes that aren't shared across multiple pods** — each replica's disk is *its own*, never a free-for-all shared volume
- **Reliable disk-backed writes and transactional consistency** (this is ACID again, at the storage layer)

Because of this, **block storage is preferred, with access mode `RWO` or `RWOP`** (`ReadWriteOncePod` — even stricter than `RWO` : guarantees the volume is mounted by at most **one specific pod**, not just one node).

The 4 concrete guarantees this translates into:

1. **Stable & predictable pod naming** — `<statefulset-name>-<ordinal>` (e.g. `mysql-0` , `mysql-1`).
2. **Stable Network Identity (DNS)** — CoreDNS-aware, covered in full in Part 4.
3. **Pods start & terminate in the right order**:
   - Start: `mysql-0 → mysql-1 → mysql-2 → mysql-3`
   - Terminate: `mysql-3 → mysql-2 → mysql-1 → mysql-0` (exact reverse)
4. **Persistent Volume re-attachment** — if `mysql-1` 's pod dies and comes back, it must reattach to **its own** volume (`mysql-data-mysql-1`), never someone else's, and never a blank one.



```
   Initial Clone  Continuous    Continuous
        |        Replication   Replication
        v             v            v
   X Replica 1    Replica 2     Replica 3
     mysql-1       mysql-2       mysql-3
  (dies here -- MUST come back as mysql-1, reattached to volume mysql-data-mysql-1, not a new pod/volu
```

What StatefulSets (STS) actually are

A Kubernetes workload controller purpose-built for managing stateful applications, where pod identity, network stability, and storage persistence are critical.

They provide:

- Stable & predictable pod naming
- Stable network identity (stable DNS name)
- Start & termination order
- Persistent volume re-attachment

These properties make STS the right tool for:

Category	Examples
Relational databases	MySQL, PostgreSQL
Distributed systems	Cassandra, Elasticsearch
Queues & coordination services	Kafka, Zookeeper
Replicated caches	Redis Cluster, MongoDB ReplicaSets

Nomenclature

- **Pod name (stable identity):** `<statefulset-name>-<ordinal-index>` → e.g. `mysql-0`
- **DNS name (stable network identity):** `<pod-name>.<headless-service-name>.<namespace>.svc.cluster.local` → e.g. `mysql-0.mysql.mysql-ha.svc.cluster.local`

PART 4 · POD-LEVEL DNS WITH A HEADLESS SERVICE

```
mysql-0     .   mysql          .   mysql-ha     . svc.cluster.local
<pod-name>      <headless-svc>     <namespace>     (K8s service domain)
```

Query	Resolvable?
`mysql-0` (bare pod name)	✗ Not resolvable

`mysql-0.mysql.mysql-ha.svc.cluster.local`	✓ Resolvable

Why a headless Service is required at all

- With a StatefulSet named `mysql` and a **headless** Service also named `mysql`, **every pod gets its own stable DNS A record**: `mysql-0.mysql.mysql-ha.svc.cluster.local`, `mysql-1.mysql.mysql-ha.svc.cluster.local`, etc. This is essential for replication, leader election, and clustering — each member needs to address the *specific* other members by name.
- **CoreDNS only generates a per-pod A record when the pod is exposed through a headless Service** (`clusterIP: None`) sharing the StatefulSet's name — there is no other path to a stable, addressable per-pod DNS name.

Why not just use a regular ClusterIP Service?

A regular ClusterIP Service **load-balances** across its backing pods — great for stateless traffic, but it **breaks deterministic routing**, which stateful apps fundamentally depend on (you need to reach *mysql-1 specifically*, not "whichever replica the load balancer picked this time").

Why not just hardcode pod names in the app instead?

StatefulSets do give you stable pod names like `mysql-0` — but that name **is not directly resolvable** unless it's exposed via a headless Service. CoreDNS does not generate A records for bare pod names on its own. The headless-Service + StatefulSet combo is the only supported way to get a genuinely resolvable, stable per-pod identity.

PART 5 · FIRST LOOK AT AMAZON EKS'S ARCHITECTURE (CONTEXT FOR THE UPCOMING DEMO 1)

EKS Control Plane

- Managed by AWS, hosted in a dedicated, AWS-owned VPC.
- Every EKS cluster has **at least 2 API server nodes** and **at least 3 etcd nodes**, spread across Availability Zones.
- AWS handles: auto-healing of faulty control-plane nodes, patching and upgrading, high availability and scalability, and security updates including certificate rotation — none of this is something you manage yourself on EKS.

EKS Data Plane

- Consists of **EC2 instances** (or optionally **Fargate**) that actually run your workloads — these live in **your own AWS account**, in a customer-managed VPC.

- Worker nodes are grouped into **Node Groups**: **Managed** Node Group or **Self-Managed** Node Group.
- Each Node Group typically uses the same AMI and instance type across its members, and can sit behind an Auto Scaling Group.

> *(Flagged for the Demo 1 session: this is purely architectural context so far — no `eksctl` /AWS console steps have been recorded yet.)*

Practice Tasks

☐ Draw your own 3-tier app and label each tier stateful or stateless — justify each label out loud rather than guessing, the way the Web Tier example in this session shows a "frontend" can still be effectively stateful.

☐ Explain the session-data failure mode (Pod 1 dies mid-session) in your own words, then explain specifically *why* moving session state to a shared cache fixes it — not just "it works," but which guarantee changed.

☐ Write out the ACID acronym from memory with a one-line example of each letter using a bank-transfer transaction.

☐ Given a StatefulSet named `mysql` with 4 replicas, write out the exact startup order and the exact (reverse) termination order without looking it up.

☐ Explain why `mysql-0` alone is unresolvable but `mysql-0.mysql.mysql-ha.svc.cluster.local` is — and why a ClusterIP Service specifically cannot substitute for the headless Service here.

What I practiced

- Worked through why a "frontend" tier can still be stateful in practice (session data pinned to a single pod) and how introducing a shared cache (ElastiCache) turns both the Web and Application tiers genuinely stateless.
- Reasoned through the primary/replica database model: initial cloning vs. continuous replication, and why this design serves both performance (read/write split) and high availability (replica promotion on primary failure).
- Learned the ACID acronym and tied each guarantee back to why plain Deployments can't support a database (no stable identity, no stable disk, no ordering guarantees).
- Learned the exact StatefulSet naming/DNS scheme (`<pod-name>.<headless-svc>.<namespace>.svc.cluster.local`) and why a headless Service — not a ClusterIP Service, not hardcoded names — is the only mechanism that makes per-pod DNS resolvable.
- Previewed the EKS control-plane/data-plane split ahead of the upcoming hands-on demo.

Merged into the master reference notes: k8s-master-notes.md §4 (Workloads) — added a "StatefulSets — Deep Dive" addendum covering the stateful-vs-stateless distinction, ACID, and the headless-Service DNS mechanism.